Практическая работа: T-SQL — управляющие конструкции, индексы и нормализация
Часть 3

Запрос:

```
USE pr1;
GO

-- ========================================
-- ПОДГОТОВКА: Добавляем заказы со статусом Pending
-- ========================================
IF NOT EXISTS (SELECT 1 FROM Orders WHERE Status = 'Pending')
BEGIN
   INSERT INTO Orders (CustomerID, OrderDate, TotalAmount, Status) VALUES
   (1, '2024-09-01', 450.00, 'Pending'),
   (2, '2024-09-05', 520.00, 'Pending');
   PRINT 'Добавлены заказы Pending';
END
GO

-- ========================================
-- ЗАДАНИЕ 1. ИНДЕКСЫ
-- ========================================
IF EXISTS (SELECT * FROM sys.indexes WHERE name = 'IX_Orders_OrderDate_Clustered') DROP INDEX
IX_Orders_OrderDate_Clustered ON Orders;
IF EXISTS (SELECT * FROM sys.indexes WHERE name = 'IX_Orders_OrderDate_NC') DROP INDEX
IX_Orders_OrderDate_NC ON Orders;
IF EXISTS (SELECT * FROM sys.indexes WHERE name = 'IX_Books_AuthorId_Covering') DROP INDEX
IX_Books_AuthorId_Covering ON Books;
IF EXISTS (SELECT * FROM sys.indexes WHERE name = 'IX_Orders_Pending_Filtered') DROP INDEX
IX_Orders_Pending_Filtered ON Orders;
GO

BEGIN TRY
   CREATE CLUSTERED INDEX IX_Orders_OrderDate_Clustered ON Orders(OrderDate);
   PRINT 'Создан кластерный индекс';
END TRY
BEGIN CATCH
   CREATE NONCLUSTERED INDEX IX_Orders_OrderDate_NC ON Orders(OrderDate);
   PRINT 'Создан некластерный индекс (CI занят PK)';
END CATCH

CREATE NONCLUSTERED INDEX IX_Books_AuthorId_Covering
ON Books(AuthorId) INCLUDE (Title, Price, StockQty);

CREATE NONCLUSTERED INDEX IX_Orders_Pending_Filtered
ON Orders(Status) WHERE Status = 'Pending';
PRINT 'Индексы созданы';
GO

/* Описание индексов:
1. Индекс на OrderDate ускоряет поиск заказов по диапазону дат.
2. Покрывающий индекс на AuthorId устраняет Key Lookup — все данные читаются из индекса.
3. Фильтрованный индекс содержит только Pending-заказы, экономя место.
*/
```

```
-- ========================================
-- ЗАДАНИЕ 2. TOP WITH TIES
-- ========================================
PRINT '--- Задание 2 ---';
SELECT TOP 3 WITH TIES Title, Price
FROM Books
ORDER BY Price DESC;
GO

-- ========================================
-- ЗАДАНИЕ 3. IF...ELSE + EXISTS
-- ========================================
PRINT '--- Задание 3 ---';
IF EXISTS (SELECT 1 FROM Books WHERE StockQty = 0)
BEGIN
    PRINT 'Внимание: есть книги с нулевым остатком!';
    SELECT Title FROM Books WHERE StockQty = 0;
END
ELSE
    PRINT 'Все книги есть в наличии';
GO

-- ========================================
-- ЗАДАНИЕ 4. TRY...CATCH + THROW
-- ========================================
PRINT '--- Задание 4 ---';
BEGIN TRY
    INSERT INTO Orders (CustomerID, OrderDate, TotalAmount, Status)
    VALUES (999, GETDATE(), 100.00, 'New');
END TRY
BEGIN CATCH
    PRINT 'Номер ошибки: ' + CAST(ERROR_NUMBER() AS NVARCHAR(10));
    PRINT 'Сообщение: ' + ERROR_MESSAGE();
    THROW;
END CATCH;
GO

-- ========================================
-- ЗАДАНИЕ 5. OUTPUT при UPDATE
-- ========================================
PRINT '--- Задание 5 ---';
UPDATE Orders
SET Status = 'Shipped'
OUTPUT deleted.OrderID, deleted.Status AS OldStatus, inserted.Status AS NewStatus
WHERE Status = 'Pending';
GO

-- ========================================
-- ЗАДАНИЕ 6. WHILE
-- ========================================
PRINT '--- Задание 6 ---';
DECLARE @threshold INT = 10;
WHILE @threshold > 0
BEGIN
    DECLARE @count INT;
    SELECT @count = COUNT(*) FROM Books WHERE StockQty < @threshold;
```

```
    PRINT 'Порог: ' + CAST(@threshold AS NVARCHAR(10)) + ', книг меньше порога: ' + CAST(@count AS
NVARCHAR(10));
    SET @threshold = @threshold - 1;
END;
GO

-- ========================================
-- ЗАДАНИЕ 7. MERGE
-- ========================================
PRINT '--- Задание 7 ---';
CREATE TABLE #BookStockStaging (
    BookId INT,
    NewStockQty INT,
    Title NVARCHAR(200),
    AuthorId INT,
    Price DECIMAL(10,2),
    ISBN NVARCHAR(13)
);

INSERT INTO #BookStockStaging (BookId, NewStockQty, Title, AuthorId, Price, ISBN) VALUES
(1, 25, 'Преступление и наказание', 1, 450.00, '9785170000001'),
(99, 50, 'Новая книга', 2, 600.00, '9785170000099');

MERGE INTO Books AS target
USING #BookStockStaging AS source
ON target.BookID = source.BookId
WHEN MATCHED THEN
    UPDATE SET StockQty = source.NewStockQty
WHEN NOT MATCHED THEN
    INSERT (Title, AuthorID, Price, StockQty, ISBN)
    VALUES (source.Title, source.AuthorId, source.Price, source.NewStockQty, source.ISBN)
OUTPUT $action, inserted.BookID, deleted.BookID;

DROP TABLE #BookStockStaging;
GO

-- ========================================
-- ЗАДАНИЕ 8. EXISTS vs IN
-- ========================================
PRINT '--- Задание 8.1 (EXISTS) ---';
SELECT DISTINCT a.FirstName, a.LastName
FROM Authors a
WHERE EXISTS (
    SELECT 1 FROM Books b
    JOIN OrderItems oi ON b.BookID = oi.BookID
    WHERE b.AuthorID = a.AuthorID
);
GO

PRINT '--- Задание 8.2 (IN) ---';
SELECT DISTINCT a.FirstName, a.LastName
FROM Authors a
WHERE a.AuthorID IN (
    SELECT b.AuthorID FROM Books b
    JOIN OrderItems oi ON b.BookID = oi.BookID
);
GO
```

```
-- ============================================
-- ЗАДАНИЕ 9. GO и область видимости
-- ============================================
PRINT '--- Задание 9. Батч 1 ---';
DECLARE @city NVARCHAR(50) = 'Москва';
SELECT * FROM Customers WHERE Address LIKE '%' + @city + '%';
GO

PRINT '--- Задание 9. Батч 2 ---';
PRINT 'Ошибка: переменная @city не объявлена';
GO


-- ============================================
-- ЗАДАНИЕ 10. Клиенты с 3+ категориями
-- ============================================
PRINT '--- Задание 10 ---';
SELECT
    c.FirstName + ' ' + c.LastName AS ClientName,
    o.OrderID,
    COUNT(DISTINCT bc.CategoryID) AS UniqueCategories
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
JOIN OrderItems oi ON o.OrderID = oi.OrderID
JOIN Books b ON oi.BookID = b.BookID
JOIN BookCategories bc ON b.BookID = bc.BookID
GROUP BY c.FirstName, c.LastName, o.OrderID
HAVING COUNT(DISTINCT bc.CategoryID) >= 3;
GO

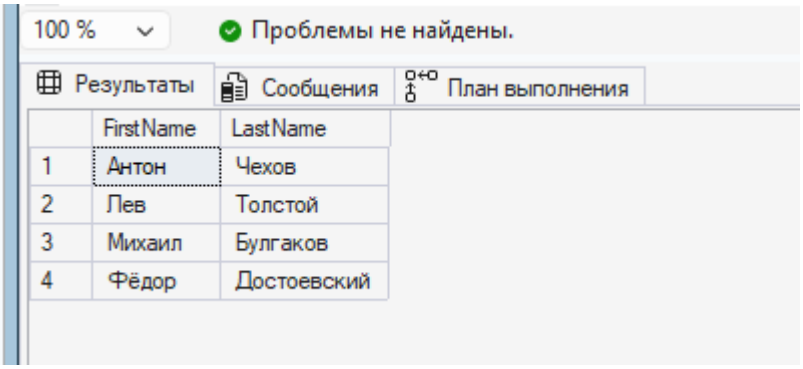

-- ============================================
-- ЗАДАНИЕ 11. Index Seek / Key Lookup
-- ============================================
```

1. До создания покрывающего индекса: Index Seek по AuthorId + Key Lookup для получения Title и Price.
2. После создания покрывающего индекса: только Index Seek без Key Lookup, так как все поля включены через INCLUDE.

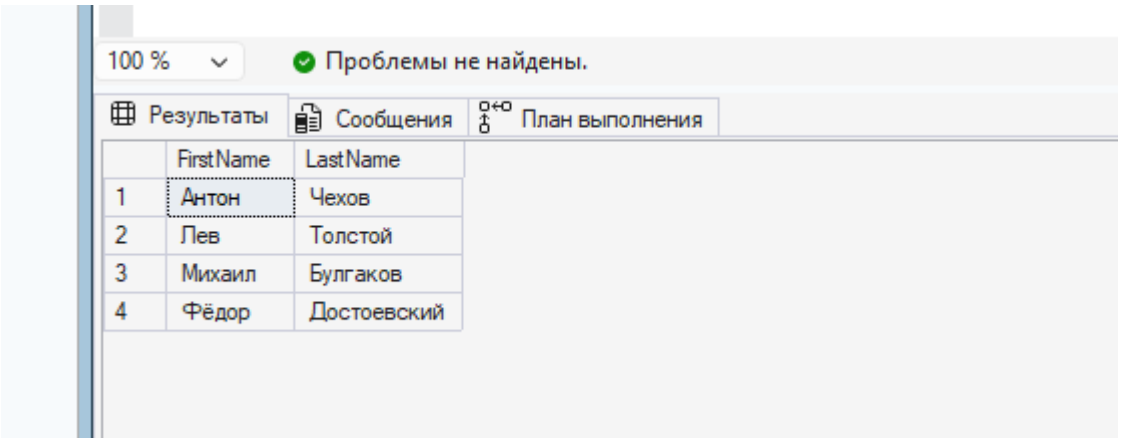
Скрины выполнения:

Задание 8

Для EXISTS:



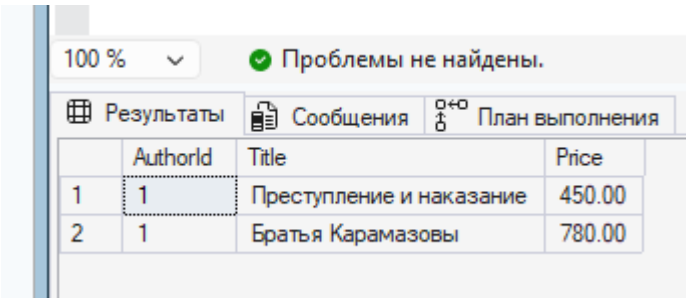
Для IN:



	FirstName	LastName
1	Антон	Чехов
2	Лев	Толстой
3	Михаил	Булгаков
4	Фёдор	Достоевский

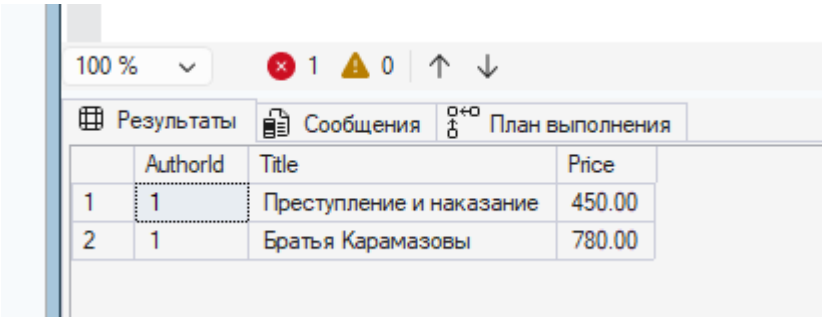
Задание 11

ДО создания индекса:



	AuthorId	Title	Price
1	1	Преступление и наказание	450.00
2	1	Братья Карамазовы	780.00

ПОСЛЕ создания индекса:



	AuthorId	Title	Price
1	1	Преступление и наказание	450.00
2	1	Братья Карамазовы	780.00